

Løkker og strenge



Program

- Løkker, Rekursion og mere om String's
- Øvelser
- Opsamling

A photograph of three women sitting at a white table in a modern office or study environment. They are all smiling and looking at their laptops. The woman on the left has blonde hair and is wearing a green top. The woman in the middle has long brown hair and is wearing a purple sweater. The woman on the right has dark hair and is wearing a grey sweater. On the table, there are two laptops, a blue water bottle, and a blue mug. Above the table, there are several potted plants, including ferns and ivy, hanging from a dark metal frame. The background is a plain white wall.

Løkker

I dag skal vi lære om **løkker**

Ikke den slags *Løkke*



Hvad er en **løkke** (eng: loop)?

Lad os starte med et eksempel:

Demo: Vejtrækningsøvelse

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("1. Træk vejret langsomt ind");  
        System.out.println("2. Hold vejret");  
        System.out.println("3. Pust langsomt ud");  
        System.out.println("4. Hold vejret");  
  
        System.out.println("1. Træk vejret langsomt ind");  
        System.out.println("2. Hold vejret");  
        System.out.println("3. Pust langsomt ud");  
        System.out.println("4. Hold vejret");  
        // ...  
    }  
}
```



```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("1. Træk vejret langsomt ind");  
        Thread.sleep(4000);  
        System.out.println("2. Hold vejret");  
        Thread.sleep(4000);  
        System.out.println("3. Pust langsomt ud");  
        Thread.sleep(4000);  
        System.out.println("4. Hold vejret");  
        Thread.sleep(4000);  
        System.out.println("1. Træk vejret langsomt ind");  
        Thread.sleep(4000);  
        System.out.println("2. Hold vejret");  
        Thread.sleep(4000);  
        System.out.println("3. Pust langsomt ud");  
        Thread.sleep(4000);  
        System.out.println("4. Hold vejret");  
        Thread.sleep(4000);  
        // ...  
    }  
}
```

Vi har lært at lave metoder, for at undgå gentagelse af kode (DRY: Don't Repeat Yourself)

```
public class Main {  
    public static void main(String[] args) {  
        breathCycle();  
        breathCycle();  
        // ...  
    }  
    public static void breathCycle() {  
        System.out.println("1. Træk vejret langsomt ind");  
        Thread.sleep(4000);  
        System.out.println("2. Hold vejret");  
        Thread.sleep(4000);  
        System.out.println("3. Pust langsomt ud");  
        Thread.sleep(4000);  
        System.out.println("4. Hold vejret");  
        Thread.sleep(4000);  
    }  
}
```

Demo: Repetition vha. løkke

```
int repetitions = 10;  
while (repetitions > 0) {  
    breathCycle();  
}
```

```
1 int repetitions = 10;  
2  
3 while (repetitions > 0) {  
4     breathCycle();  
5     repetitions = repetitions - 1;  
6 }
```



```
1 int repetitions = 10;  
2  
3 while (repetitions > 0) {  
4     breathCycle();  
5     repetitions = repetitions - 1;  
6 }
```

```
1 int repetitions = 10;  
2  
3 while (repetitions > 0) {  
4     breathCycle();  
5     repetitions--;  
6 }
```

```
1 int repetitions = 10;  
2  
3 while (repetitions > 0) {  
4     breathCycle();  
5     repetitions--;  
6 }
```

A photograph of two men in a modern office setting. The man on the left has a full beard and is wearing a dark jacket over a plaid shirt. The man on the right is wearing a plaid shirt and is looking at a laptop. They are sitting at a desk with multiple computer monitors displaying code. A large window in the background shows a cityscape with buildings. A white rectangular box with the word 'Anvendelser' is overlaid on the image.

Anvendelser

Vi kan bruge en løkke til:

- at gøre noget et **antal gange**
- at gøre noget **indtil** en bestemt tilstand er opnået
- at gennemløbe en **liste** (fx en String)
- at blive ved med at spørge brugeren om input, indtil det er gyldigt

Input-validering vha. en løkke

```
Scanner scanner = new Scanner(System.in);
int age = -1;
while (age < 0 || age > 100) {
    System.out.print("Indtast din alder: ");
    age = scanner.nextInt();
}
System.out.println("Din alder er: " + age);
```

Så længe brugeren indtaster en alder, der er mindre end 0 eller større end 100, bliver der ved med at spørge om en ny alder, dvs.

Indtast din alder: 101 Indtast din alder: -5 Indtast din alder: 44 Din alder er: 44

En **String** er en liste af **char**-værdier

```
String name = "Lars";  
char firstChar = name.charAt(0); // 'L'  
char secondChar = name.charAt(1); // 'a'  
char thirdChar = name.charAt(2); // 'r'  
char fourthChar = name.charAt(3); // 's'
```


Vi kan bruge en løkke til at gennemløbe en String

```
String name = "Lars";  
int length = name.length();  
int index = 0;  
  
while (index < length) {  
    char c = name.charAt(index);  
    System.out.println(c);  
    index++;  
}
```

Uendelige løkker

Demo: Uendelig løkke

```
while (true) {  
    breathCycle();  
}
```

Vi kan stoppe programmet med en **Ctrl + C** i terminalen eller ved at stoppe det i IntelliJ

Eller vi kan stoppe med et **break**-statement:

```
int repetitions = 10;
while (true) {
    breathCycle();

    if (repetitions == 0) {
        break;
    }

    repetitions = repetitions - 1;
}
```

En uendelig løkke kan være nyttig, fx

- i spil, hvor spillet skal køre indtil brugeren lukker det ned
- i servere, der skal køre og lytte efter beskeder hele tiden

Der er andre måder at ende i en uendelig løkke på

```
public static void breathCycle() {  
    System.out.println("1. Træk vejret langsomt ind");  
    System.out.println("2. Hold vejret");  
    System.out.println("3. Pust langsomt ud");  
    System.out.println("4. Hold vejret");  
  
    breathCycle(); // rekursion  
}
```


A woman with dark curly hair tied back, wearing a dark jacket with white stitching, stands on a balcony. She is looking out over a city skyline with various buildings and a church spire in the background. The scene is bright and sunny.

Rekursion

Når en metode kalder sig selv

```
public static void countDown(int n) {  
    if (n <= 0) {  
        System.out.println("Liftoff!");  
    }  
  
    System.out.println(n);  
    countDown(n - 1);  
}
```

som vi kan kalde med

```
countDown(10);
```



Tre ting du tager med fra fra i dag?